

MOTİFİKA

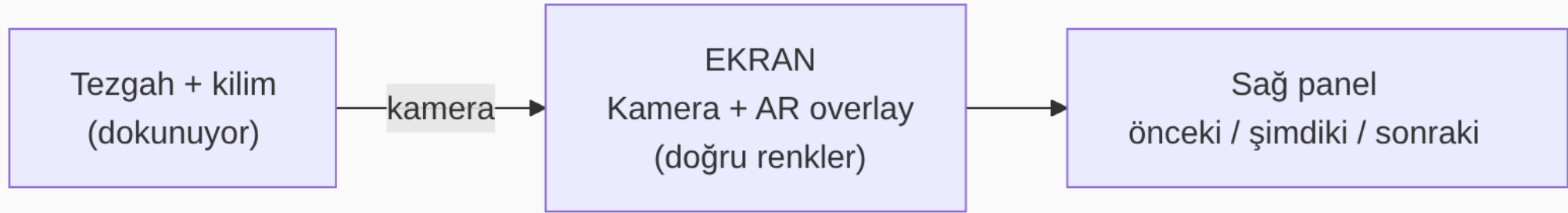
Kamerayla kilim dokuma rehberi

Doğru sıradaki motifi ekranda gösteren AR sistemi

Raspberry Pi · OpenCV · Python · Homography tabanlı AR overlay

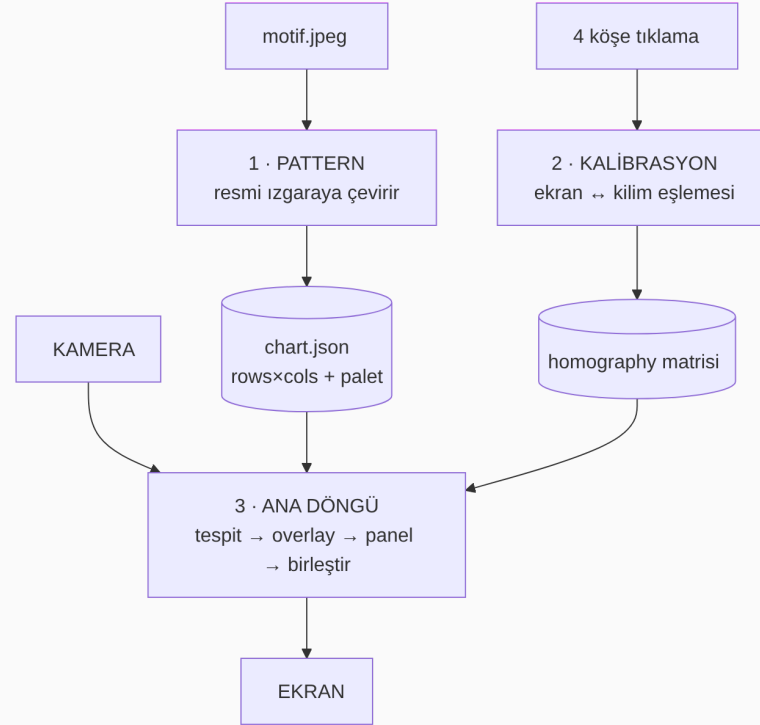
Proje tek cümlede

Kullanıcı kilim dokurken, kameraya bakan sistem **dokunan sırayı otomatik bulur**, motifin o sıradaki **doğru renklerini ekrana yansıtır** ve **bir sonraki sırayı** önceden gösterir.



- **Tespit etmez, gösterir:** "yanlış yaptın" demek yerine *doğru olanı* üst üste bindirir.
- Tetris'in "sonraki parça" mantığı: panelde **önceki / şimdiki / sonraki** sıra.

Sistem mimarisi — kuşbakışı



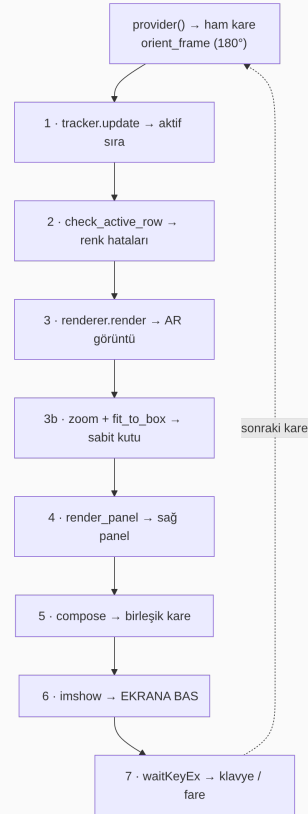
İki şey **bir kez** hazırlanır (chart + kalibrasyon), gerisi **her karede** tekrar eder.

Modül haritası — app/ klasörü

Modül	Görevi	Anahtar fonksiyon
pattern.py	Motif resmi → ızgaralı chart (k-means palet)	build_chart()
calibration.py	4 köşe → homography matrisi	compute_homography()
progress.py	Hangi sıra dokunuyor? atkı cephesi tespiti	ProgressTracker.update()
overlay.py	Chart'ı kameraya AR olarak yansıt	OverlayRenderer.render()
color_check.py	Sıradaki renkleri palet ile karşılaştır	check_active_row()
ui.py	Sağ paneli çiz + ekranı birleştir	UIRenderer.compose()
main.py	Hepsini sırayla bağlayan ana döngü	main()

Omurga: **homography** — chart koordinatı ile kamera pikseli arasındaki köprü. 4 modül onu kullanır.

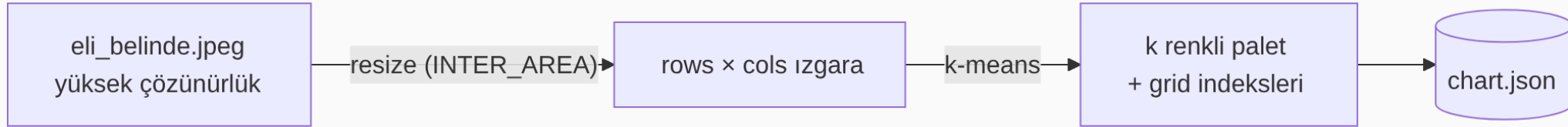
Veri akışı — her karede ne oluyor?



Kaynak: `app/main.py` · `while True:` döngüsü

ADIM 0 — Motif resmi nasıl ızgaraya çevriliyor?

pattern.py üç işlem yapar: **küçült** → **renkleri azalt** → **kaydet**



```
def build_chart(image_path, rows, cols, palette_size):
    raw = cv2.imread(str(image_path), cv2.IMREAD_UNCHANGED)
    bgr = flatten_alpha(raw) # PNG saydamlığını beyaza düzleştir
    small = pixelate(bgr, rows, cols) # her hücre = 1 piksel olacak şekilde küçült
    palette_rgb, grid = quantize_palette(small, palette_size) # k-means ile renkleri azalt
    return {"rows": rows, "cols": cols,
            "palette": palette_rgb.tolist(), # [[r,g,b], ...]
            "grid": grid.tolist()} # [[palet_idx, ...], ...] → rows × cols
```

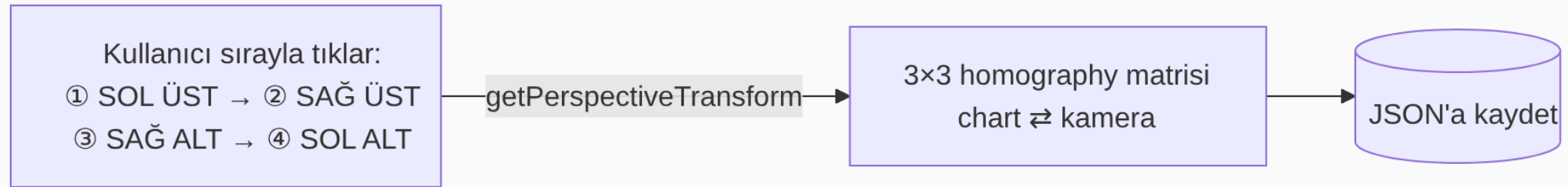
ADIM 0 — Izgaraya çevirme: detay

```
def pixelate(img_bgr, rows, cols):  
    # cv2.resize (W, H) ister → (cols, rows). INTER_AREA küçültmede en temiz sonuç.  
    return cv2.resize(img_bgr, (cols, rows), interpolation=cv2.INTER_AREA)  
  
def quantize_palette(small_bgr, k):  
    pixels = small_bgr.reshape(-1, 3).astype(np.float32) # her piksel bir örnek  
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 20, 0.5)  
    _, labels, centers = cv2.kmeans(pixels, k, None, criteria,  
                                   attempts=5, flags=cv2.KMEANS_PP_CENTERS)  
    palette_rgb = np.clip(centers, 0, 255).astype(np.uint8)[: , ::-1] # BGR→RGB  
    grid = labels.reshape(rows, cols).astype(int) # her hücre → palet indeksi  
    return palette_rgb, grid
```

- **Neden k-means?** Fotoğrafta binlerce renk var; dokumada ise sadece birkaç iplik rengi. K-means en baskın **k** rengi bulup geri kalanı bunlara yuvarlar.
- Sonuç **bir kez** üretilir, `assets/<motif>_chart.json` olarak saklanır (cache).

ADIM 1 — Kalibrasyon: ekranı kilime oturtmak

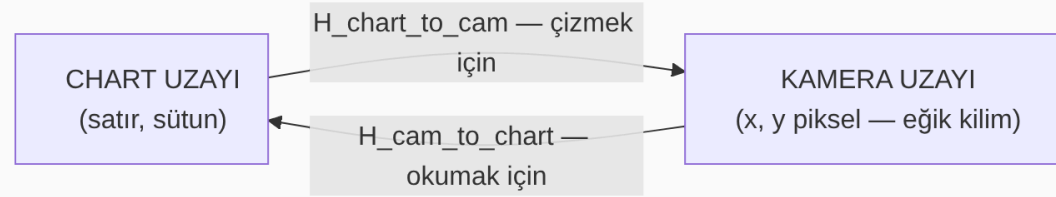
Kamera kilimi eğik/uzaktan görür. Sistem "ekrandaki ızgara, kilimde tam nereye düşüyor?" sorusunu **4 köşe tıklamasıyla** çözer.



```
def compute_homography(camera_corners, rows, cols):  
    src_chart = chart_corners(rows, cols)      # [[0,0],[cols,0],[cols,rows],[0,rows]]  
    dst_cam   = camera_corners                 # tıklanan 4 kamera pikseli  
    H_chart_to_cam = cv2.getPerspectiveTransform(src_chart, dst_cam)  
    H_cam_to_chart = np.linalg.inv(H_chart_to_cam) # ters yön  
    return H_chart_to_cam, H_cam_to_chart
```


Homography nedir? — sistemin omurgası

Bir düzlemdeki noktayı başka bir düzleme taşıyan **3×3 matris**. 4 köşe eşleşmesi tam yeter (8 bilinmeyen = 4 nokta × 2 koordinat).



Yön	Kullanan modüller	Amaç
H_cam_to_chart	progress.py , color_check.py	kamerayı oku (düzleştir)
H_chart_to_cam	overlay.py	chart'ı kameraya çiz

ADIM 2 — Kamera nasıl açılıyor?

`open_camera_or_image()` hem canlı kamera hem de sabit görüntü (test) için tek arayüz döndürür: `provider()` fonksiyonu her çağrıda taze bir kare verir.

```
def open_camera_or_image(args, frame_size_hint=(1280, 720), flip=True):
    cap = cv2.VideoCapture(args.camera)          # kamerayı aç (indeks: 0, 42, ...)
    if not cap.isOpened():
        raise RuntimeError(f"kamera açılmadı: {args.camera}")
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, frame_size_hint[0]) # 1280 iste
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, frame_size_hint[1]) # 720 iste

    def provider():                               # her çağrıda taze bir kare verir
        ok, frame = cap.read()
        if not ok:
            return None
        return orient_frame(frame) if flip else frame # 180° döndür (kullanıcı bakışı)

    return provider, cap, frame_size
```

`orient_frame = cv2.rotate(frame, ROTATE_180)`: kamera projeyi baş aşağı gördüğü için döndürülür. Çeviri **kaynakta** yapılır → tespit, overlay ve ekran hep aynı yöne bakar.

ADIM 3 — Ana döngü (basitleştirilmiş)

```
while True:
    frame = provider()                # 1) kameradan kare al
    if frame is None: break

    active_row = tracker.update(frame, H_cam_to_chart) # 2) hangi sıra dokunuyor?

    if frame_idx % COLOR_CHECK_EVERY_N == 0:          # 3) renk kontrolü (her 5 karede 1)
        last_mismatches = check_active_row(frame, H_cam_to_chart, chart, check_row, backend)

    ar_view = renderer.render(frame, H_chart_to_cam, active_row) # 4) AR overlay çiz
    ar_view = _fit_to_box(_zoom_view(ar_view, zoom), *camera_box) # sabit kutuya oturt

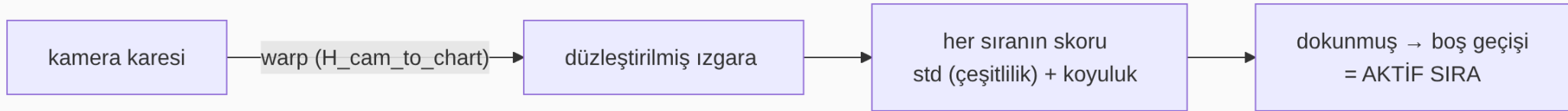
    panel = ui.render_panel(chart, active_row, direction, last_mismatches, ...) # 5) panel
    composed = ui.compose(ar_view, panel, vertical=args.portrait) # 6) yan yana birleştir

    cv2.imshow(win, composed)          # 7) EKRANA BAS
    key = cv2.waitKeyEx(1)             # 8) klavye / fare oku
    # ... ok tuşları, zoom, saydamlık, çıkış ...
```

Her satır bir modülü çağırır. Döngünün tek görevi: **sırayı korumak** ve sonuçları birbirine geçirmek.

ADIM 3.1 — Atkı cephesi tespiti (hangi sıra?)

`progress.py` kamerayı düzleştirip her sıranın "dokunmuşluk skorunu" hesaplar; dokunmuş→boş geçişini aktif sıra kabul eder.

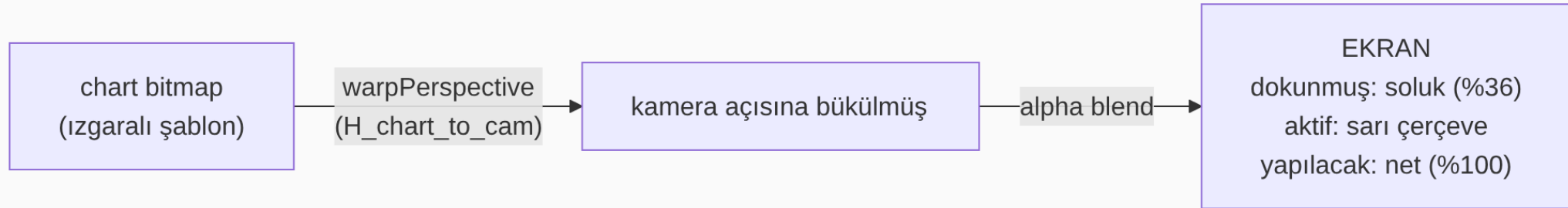


```
def row_scores(self, warped_bgr):  
    gray = cv2.cvtColor(warped_bgr, cv2.COLOR_BGR2GRAY)  
    for r in range(self.rows):  
        band = gray[r*h:(r+1)*h]  
        std_h = band.std() # dokunmuş bölge renk açısından çeşitli  
        darkness = 1.0 - band.mean()/255 # dokunmuş iplik koyu  
        scores[r] = 0.6*std_h/80 + 0.4*darkness # karma skor
```

EMA ile yumuşatma + relatif eşik (ışıktan bağımsız) titremeyi önler. Kullanıcı **ok tuşlarıyla** elle düzeltebilir (`tracker.bump`).

ADIM 3.2 — AR overlay (chart'ı kameraya yansıt)

`overlay.py` chart'ı bir kez bitmap'e çevirir, her karede homography ile kameraya **warp** eder ve üç bölgeyi farklı saydamlıkla harmanlar.



```
def render(self, frame_bgr, H_chart_to_cam, active_row):
    warped_layer = cv2.warpPerspective(self._chart_layer, M, out_size) # chart'ı bükür
    alpha = self._alpha_mask(active_row) # bölgelere saydamlık
    warped_alpha = cv2.warpPerspective(alpha, M, out_size)[..., None]
    blended = warped_layer*warped_alpha + frame_bgr*(1 - warped_alpha) # piksel piksel harmanla
    out = np.clip(blended, 0, 255).astype(np.uint8)
    self._draw_active_row_border(out, H_chart_to_cam, active_row) # aktif sıraya sarı kutu
    return out
```

ADIM 3.3 — Renk kontrolü (sıra doğru mu?)

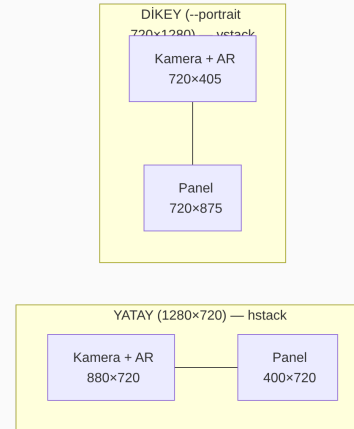
`color_check.py` tamamlanan sıranın her hücresinin rengini örnekler ve **LAB renk uzayında** en yakın palet rengini bulur. Beklenenle uyuşmazsa uyarı.

```
def check_active_row(frame_bgr, H_cam_to_chart, chart, active_row, backend):
    samples = sample_active_row(frame_bgr, H_cam_to_chart, ...) # her hücrenin orta rengi
    observed_idx, dist = backend.classify(samples) # en yakın palet rengi (LAB)
    expected_idx = chart.grid[active_row] # chart'ta olması gereken
    mismatches = []
    for c in range(chart.cols):
        if observed_idx[c] != expected_idx[c]: # uyuşmuyor mu?
            mismatches.append((c, expected_idx[c], observed_idx[c], dist[c]))
    return mismatches # → panelde "S5.12: Kırmızı yerine Siyah"
```

- **Neden LAB?** LAB uzayında iki renk arasındaki Öklid mesafesi $\approx$ gözün gördüğü fark.
- **Neden her 5 karede 1?** Kontrol pahalı; gözle anlık görünür ama CPU'yu yormaz.
- HSV/LAB **fallback** her zaman çalışır; ileride Hailo-8L NPU'ya (`HailoBackend`) yükseltilebilir.

ADIM 4 — Ekran nereden oluşuyor? (ui.py)

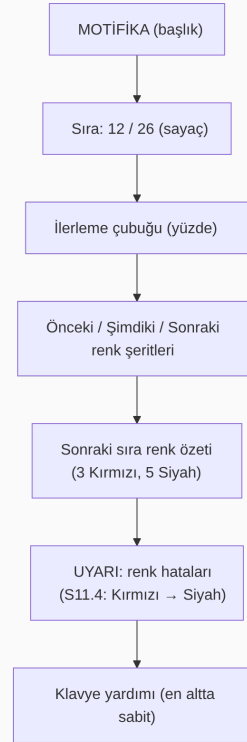
İki parça ayrı çizilir, sonra `compose()` ile birleştirilir.



```
def compose(self, camera_view, panel, vertical=False):  
    if vertical:                                     # portrait: alt alta  
        return np.vstack([camera_view, panel])  
    return np.hstack([camera_view, panel])           # landscape: yan yana
```

ADIM 4 — Sağ panel ne içeriyor?

`render_panel()` koyu zeminli bir görüntüye yukarıdan aşağı bileşenleri çizer:



Türkçe karakterler (ı, ş, ğ, ç) için metinler **PIL + DejaVu TTF** ile çizilir — `cv2.putText` Hershey fontu Türkçe'yi "?" gösterir.

Kontroller — klavye ve dokunmatik

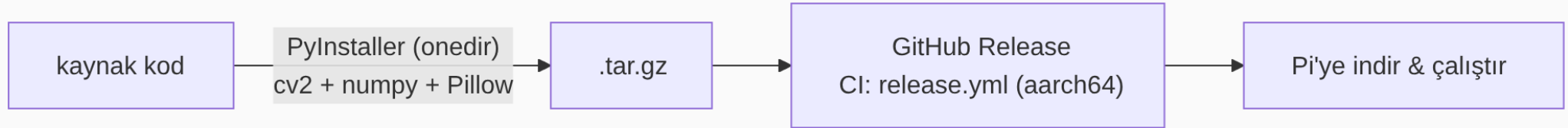
Tuş	İşlev	Ekran butonu	İşlev
↑ / ↓	aktif sırayı elle kaydır	Saydamlık -/+	overlay görünürlüğü
z / x	dijital yakınlaş / uzaklaş	Motif	kalibrasyonu koruyarak motif değiştir
+ / -	saydamlık (şeffaf ↔ opak)		
r	yeniden kalibre et		
d	dokuma yönü (alttan-üst ↔ üstten-alt)		
c	renk kontrolü aç / kapat		
q / ESC	çıkış		

Ok tuşları `waitKeyEx` ile okunur — `waitKey()&0xFF` ok kodlarını ASCII harflere çakıştırıyordu (Yukarı → 'R' = yanlışlıkla kalibrasyon!).

Çalıştırma ve dağıtım

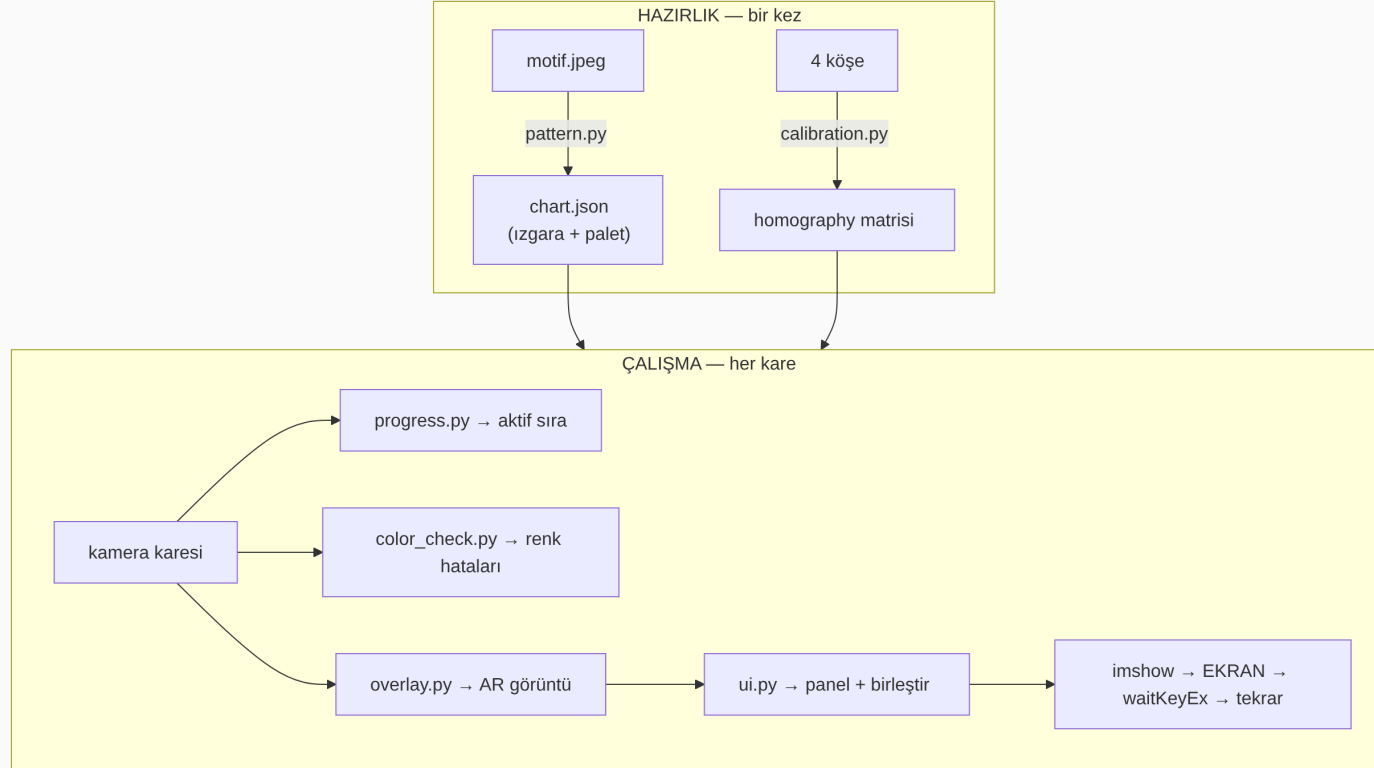
```
# Geliştirme (laptop webcam):  
python -m app.main --motif eli_belinde --rows 30 --cols 60 --camera 0  
  
# Sabit görüntü ile test (kameronasız):  
python -m app.main --motif hayat_agaci --rows 50 --cols 30 --image kilim.jpg  
  
# Raspberry Pi (tam ekran + kamera servisi):  
scripts/run.sh --motif eli_belinde --rows 30 --cols 60
```

Dağıtım: Pi'de Python kurulu değil → uygulama **PyInstaller** ile tek pakete dondurulur, **GitHub Release**'e `.tar.gz` olarak yüklenir.



Çalışma-zamanı bağımlılıkları yalın: `opencv-python` (GUI sürümü, headless değil) · `numpy` · `Pillow`.

Özet — uçtan uca akış



- **Tespit etmek yerine göstermek:** eğitim verisi gerekmez, demo güvenilir.
- **Homography** her şeyi birbirine bağlar: oku (cam→chart) ve çiz (chart→cam).
- Modüler yapı: her dosya tek iş yapar, `main.py` orkestra şefi.

Teşekkürler

Sorular?

MOTİFİKA · Python + OpenCV · Raspberry Pi · 2026